

Python Data Pipeline Audit

15-Minute Practical Quiz

Read the case, then answer all 5 questions.

Case: Cleaning Raw Order Logs

Maya is an Operations Analyst at FreshBite Online Grocery. Every evening, the web server outputs a text-based comma-separated log file containing raw customer orders. Maya needs to write a simple Python script to process these lines, compute total sales revenue for valid orders, and discard corrupted or canceled entries.

Line 1 — "ORD-101, STORE_NORTH, 34.50, DELIVERED\n"
Line 2 — "ORD-102, STORE_SOUTH, INVALID_PRICE, DELIVERED\n"
Line 3 — "ORD-103, STORE_NORTH, 12.00, CANCELLED\n"
Line 4 — "ORD-104, STORE_EAST, 89.99, DELIVERED\n"

Maya notices that each line comes in as a single text string (`str`) containing trailing newline characters (`\n`). To process an order, the status must be exactly "DELIVERED", and the order amount must be successfully parsed into a floating-point number (`float`).

Key System Specs

- **Daily Log Size:** ~100 order lines per day
- **Required Output:** Total revenue grouped by store ID
- **Environment:** Standard Python 3.x using native data structures (`list`, `dict`, `tuple`)

Quiz

Question 1 (MCQ)

Given the raw string `line = "ORD-101, STORE_NORTH, 34.50, DELIVERED\n"`, which expression correctly removes whitespace and returns a list of 4 clean string elements?

- ☐ (A) `line.strip().split(",")`
- ☐ (B) `line.replace("\n").split()`
- ☐ (C) `float(line).split(",")`
- ☐ (D) `line.trim().tolist(",")`

Question 2 (MCQ)

Maya wants to store cumulative revenue totals per store (e.g., {"STORE_NORTH": 34.50, "STORE_EAST": 89.99}). Which Python data structure and operation should she use to accumulate these values dynamically?

- ☐ (A) A `list`, appending amounts using `.append()`
- ☐ (B) A `dict`, keying by store ID and adding amounts to running values
- ☐ (C) A `tuple`, updating values with `.insert()`
- ☐ (D) A `set`, storing store IDs and amounts as unordered pairs

Question 3 (MCQ)

When processing Line 2 (`Amount = "INVALID_PRICE"`), executing `float("INVALID_PRICE")` will raise a runtime `ValueError` and crash the script. What is the most Pythonic way to handle this error safely?

- ☐ (A) Wrap the conversion in a `try...except ValueError:` block to skip bad lines
- ☐ (B) Call `sys.exit()` to abort the script when a string is encountered
- ☐ (C) Convert the string using `int("INVALID_PRICE")` instead
- ☐ (D) Delete the file before running the script

Question 4 – Written (Code Construction)

Write a short Python function `parse_order(raw_line)` that takes a single log string (e.g., "ORD-101, STORE_NORTH, 34.50, DELIVERED\n").

The function should:

- (1) Strip trailing spaces/newlines and split the string by commas.
- (2) Check if the status is "DELIVERED".
- (3) Convert the amount to `float` using a `try...except` block.
- (4) Return a tuple (`store_id`, `amount`) if valid, or `None` if invalid/cancelled.

Question 5 – Written (Conceptual Analysis)

In 2–3 sentences, explain why using explicit `try...except` blocks (Exception Handling) is superior to writing complex nested `if...else` statements when parsing messy external data files in production.
